

OWASP Top 10

Cryptographic Failures

BIG school

BIGSEIO

GENERAMOS
NEGOCIO

Cryptographic Failures

Índice

- Introducción
- ¿Qué significa "Cryptographic Failures"?
- ¿Por qué ocurre esta vulnerabilidad?
- Ejemplos reales y típicos
- ¿Qué puede hacer un atacante si explota esto?
- ¿Cómo detectarlo?
- ¿Cómo prevenirlo?
- ¿Cómo mitigar si ya estás afectado?
- Checklist rápido
- Conclusiones



Cryptographic Failures

BIG school

BIGSEIO

GENERAMOS
NEGOCIO

Introducción

- La criptografía protege confidencialidad, integridad y autenticidad de los datos
- No basta con “añadir cifrado”: debe implementarse con estándares y parámetros correctos
- Objetivo: entender fallos comunes, su impacto real y cómo aplicar criptografía moderna de forma segura

¿Qué significa "Cryptographic Failures"?

Errores en diseño, uso, configuración o gestión de controles criptográficos

- Expone datos sensibles o permite su manipulación sin que el sistema lo detecte
- Incluye algoritmos obsoletos, mala gestión de claves y transporte sin validación
- No es solo "usar cifrado", es usarlo con protocolos, librerías y prácticas validadas

¿Por qué ocurre esta vulnerabilidad?

- Uso de algoritmos heredados o débiles (MD5, SHA-1, DES, RC4)
- Criptografía personalizada ("inventada") en lugar de estándares probados
- Claves hardcodeadas, sin rotación o almacenadas en texto plano/configs
- Cifrado sin autenticación (modos ECB/CBC sin MAC) que permite alteraciones silenciosas
- Falta de validación TLS o aceptación de certificados no verificados en clientes

Ejemplos reales y típicos

- Contraseñas en texto plano o con hashing rápido (MD5/SHA-256 sin salt)
- Cifrado simétrico sin verificar integridad: datos modificables sin detección
- Secretos hardcodeados en repositorios (un commit filtrado compromete todo el entorno)
- TLS mal configurado: versiones antiguas, suites débiles o validación desactivada
- Descarga de artefactos/dependencias sin firma digital o checksum de verificación

¿Qué puede hacer un atacante si explota esto?

- Robo masivo de credenciales y acceso no autorizado a cuentas o sistemas
- Exposición de datos sensibles (personales, financieros, de salud o corporativos)
- Manipulación silenciosa de información sin que la aplicación lo detecte
- Intercepción de comunicaciones mediante ataques de intermediario (MITM)
- Compromiso de pipelines o infraestructura si se alteran artefactos no firmados

¿Cómo detectarlo?

- **Desarrollo:** SAST para identificar algoritmos débiles, claves expuestas o modos inseguros
- **Auditoría:** Escanear repositorios/buckets en busca de secretos; revisar configuración TLS y suites de cifrado
- **Producción:** Monitorear accesos a KMS/Vault, errores criptográficos repetitivos o certificados por caducar
- **Señales:** Fallbacks a protocolos antiguos, uso de funciones de hashing obsoletas, secretos en logs o configs

¿Cómo prevenirlo?

- No reinventar: usar librerías mantenidas y validadas por la comunidad (OpenSSL, libsodium, etc.)
- Contraseñas: Argon2id, bcrypt o PBKDF2 con salt único por usuario
- Cifrado en reposo: modos autenticados (AES-GCM, ChaCha20-Poly1305), nunca reutilizar IVs/nonces
- Transporte: forzar TLS 1.2/1.3, deshabilitar suites débiles, validar certificados del servidor
- Gestión de secretos: KMS/Vault con rotación automática y acceso por mínimo privilegio
- Integridad: firmar artefactos/imágenes y verificar firmas antes de desplegar o ejecutar

¿Cómo mitigar si ya estás afectado?

- Determinar alcance: identificar qué datos, claves o servicios están comprometidos
- Rotar inmediatamente: claves, secretos, tokens y certificados potencialmente expuestos
- Invalidar sesiones y forzar reautenticación si dependen de la clave afectada
- Actualizar librerías, deshabilitar algoritmos inseguros y habilitar estándares modernos
- Auditar logs y tráfico para detectar extracción o manipulación previa al cierre
- Aplicar controles compensatorios: MFA obligatorio, restricciones de red o cifrado adicional
- Notificar según normativas y documentar para automatizar validaciones futuras

Checklist rápido

1. ¿Se usan algoritmos modernos y validados (Argon2id, AES-GCM, TLS 1.3)?
2. ¿No hay claves, tokens ni contraseñas hardcodeadas en código o configuraciones?
3. ¿TLS 1.2/1.3 está habilitado y se validan certificados correctamente?
4. ¿El cifrado es autenticado (AEAD) y los IVs/nonces son únicos por operación?
5. ¿Las contraseñas usan hashing seguro con salt único por usuario?
6. ¿Los artefactos se firman y verifican antes de usar o desplegar?
7. ¿Hay rotación programada de claves y secretos gestionados por KMS/Vault?

Conclusiones

- La criptografía es poderosa pero frágil: implementarla mal es peor que no usarla
- Confía en estándares modernos, nunca en soluciones personalizadas o heredadas
- Protege claves y secretos con la misma urgencia que proteges el código fuente
- Automatiza detección, rotación y verificación de integridad en CI/CD
- Como desarrollador: aplicar buenas prácticas criptográficas es proteger a los usuarios incluso cuando nadie mira



¡MUCHAS GRACIAS!

BIG school

BIGSEIO

**GENERAMOS
NEGOCIO**